

SOFTWARE TESTING · AI-FIRST SEMINAR TRACK (2026)

T07 — CI/CD & Test-Harness Engineering

“Tests are useless if they don't run automatically.” Pipelines + AI flaky-test triage.

Fundamentals

GitHub Actions

Live demo

Flaky + AI triage

2-hour session · SUT: EShop · primary CI: GitHub Actions

A test no one runs doesn't exist

Manual test runs rot. CI makes every push prove itself — and AI now triages the failures.

0–30'

Fundamentals of CI/CD & the test harness

30–55'

GitHub Actions — workflow, YAML, tooling compare

55–75'

Live demo — EShop pipeline on every PR

75–95'

Flaky tests: taxonomy + AI vs human triage

95–115'

Activity — Pipeline Postmortem

115–120'

Wrap-up — where AI helps vs adds noise

By the end of this session you can...

- › **Author a GitHub Actions workflow** — build → test → artifact upload, running on every PR.
- › **Compare CI engines** — run the same flow on Jenkins / GitLab CI and note the differences.
- › **Diagnose a flaky test** — introduce one, then analyse it manually with logs + retry history.
- › **Use AI to surface the same flake** — ChatGPT/Claude on logs, or Datadog Flaky Test Management.
- › **Judge AI critically** — decide where AI adds value in CI and where it adds noise.

CI, CD, and CD — one feedback loop

Shrink the gap between writing code and knowing it works.



Continuous Integration

Every push is merged & tested automatically. Fast feedback, no “works on my machine”.



Continuous Delivery

Every green build is always releasable — one click to ship to staging/prod.



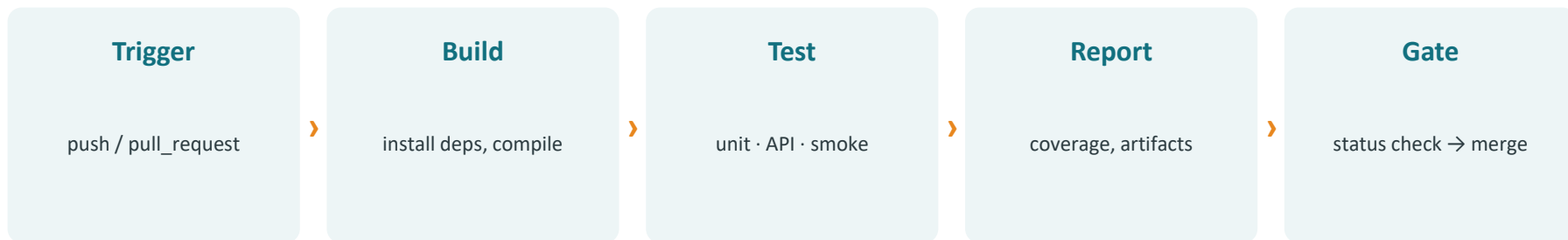
Continuous Deployment

Every green build ships to production automatically — no human gate.

KEY IDEA This seminar lives in CI — the layer that runs your tests. Delivery/deployment are the next step.

Anatomy of a pipeline

A trigger fires stages; each stage can fail the build and block the merge.



FAIL FAST Order stages cheap→expensive: lint before unit before API before e2e. A red stage stops the pipeline early and saves minutes.

ARTIFACTS Coverage XML, test reports, screenshots — saved on every run so a failure is debuggable after the fact.

The test harness on every PR

Layer the tests: many fast unit tests, fewer API tests, a handful of smoke checks.

THE PYRAMID, WIRED INTO CI

- › **Unit** — pure logic, milliseconds, run on every push.
- › **API / integration** — routes + DB, run on every PR.
- › **Smoke** — a few end-to-end happy paths before merge.
- › **Coverage** — emit XML; post a badge on the PR.

THE GATE

Status check

green ✓ required to merge

Branch protection

main can't be pushed to directly; PR + passing

Result

broken code never reaches the main branch

The mental model

Events trigger workflows; workflows contain jobs; jobs run steps on runners.

- › **Event** — push, pull_request, schedule, manual dispatch.
- › **Workflow** — a YAML file in .github/workflows/ .
- › **Job** — a set of steps on one runner (ubuntu-latest).
- › **Step** — a shell command or a reusable action (uses:).
- › **Runner** — the VM/container; GitHub-hosted or self-hosted.

WHY START HERE

62%

personal projects

41%

of organizations

YAML-only · free for
public repos

JetBrains State of
CI/CD 2025

A workflow for EShop — build & test on every PR

ci.yml

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request: { branches: [main] }
  push: { branches: [main] }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20,
                  cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage
```

READ IT TOP-DOWN

- › **on:** — runs on PR and on push to main.
- › **runs-on:** — a fresh Ubuntu VM each run.
- › **uses:** — pinned reusable actions (@v4).
- › **cache: npm** — skip re-downloading deps.
- › **run:** — lint, then Jest with coverage.

Artifacts, matrix & a coverage badge

add-ons

```
# upload coverage as an artifact
- uses: actions/upload-artifact@v4
  with:
    name: coverage
    path: coverage/clover.xml

# run across Node versions in parallel
strategy:
  matrix:
    node: [18, 20, 22]
runs-on: ubuntu-latest

# gate: this job is the required check
#   Settings → Branches → protect main
```

WHAT EACH BUYS YOU

- › **Artifacts** — download logs/coverage from any run.
- › **Matrix** — one file, many OS/runtime combos.
- › **Badge** — coverage % shown on the PR & README.
- › **Protection** — make this job required to merge.

Pick one as your main — know the trade-offs

Tool	Model	Best when	Watch out
GitHub Actions	YAML, hosted	Public repos, fast start, marketplace actions	Minutes cost on private repos at scale
Jenkins	Self-hosted	Full control, mature plugins, on-prem	You maintain the server & plugins
GitLab CI	Built-in	Secrets/environments tightly integrated	Best inside the GitLab ecosystem

REALITY 32% of orgs run two CI tools, 9% run three (JetBrains 2025). Portability of your test scripts matters more than the engine.

What we're about to run

EShop: one PR triggers the whole harness — watch it go red, then green.

1

Open a PR

Push a branch; the CI check appears as “pending”.

2

Pipeline runs

Lint → Jest API tests → coverage
XML uploaded.

3

See the gate

Red check blocks merge; fix;
green check unblocks.

4

Read artifacts

Download coverage; open the
badge on the PR.

Now break it on purpose

Introduce one sleep-based flake; run 10× and watch it fail intermittently.

a sleep-based flake

```
// api/checkout.test.js
test("order confirms within 200ms", async () => {
  const t0 = Date.now();
  await placeOrder(cart);
  // flaky: real latency varies 120-260ms
  await sleep(50);           // race!
  const order = await getLatest();
  expect(order.status).toBe("CONFIRMED");
  expect(Date.now() - t0).toBeLessThan(200);
});
```

OBSERVE OVER 10 RUNS

- › Passes ~7/10, fails ~3/10 — no code changed.
- › Same commit, different result = flaky by definition.
- › Timing assertion + fixed sleep = the classic race.
- › Retry history is the first evidence to collect.

The tax you pay for ignoring them

A flaky test passes or fails on the same code — eroding trust in the whole suite.

WHY THEY HURT

- › Developers start ignoring red — real bugs slip by.
- › Re-runs waste CI minutes and wall-clock time.
- › “Just hit retry” becomes the culture.

~20%

of CI time is spent on flaky tests and their re-runs across real-world pipelines.

Industry analyses, 2025

Flake taxonomy — name it before you fix it

AI triage only works if you can classify the failure. Three big families:

Timing

Races, fixed sleeps, timeouts, animations, clock/timezone assumptions.

FIX: await a condition, not a duration

Ordering

Tests share state; pass alone, fail in a suite; leaked DB rows/globals.

FIX: isolate + reset state per test

Environment

CPU load, network, missing env var, container vs local, time-of-day.

FIX: pin versions; mock the network

Manual triage — the hypothesis tree

Humans are good at pruning: from symptom to a small set of likely causes.

from logs + retry history

```
FAIL: order confirms within 200ms (3/10 runs)
```

Hypothesis tree

```
├─ timing?
│   ├── fixed sleep(50) too short ← likely
│   └─ CI runner slower than laptop ← likely
├─ ordering?
│   └─ prior test leaves open order ← check
└─ environment?
    └─ DB container cold-start ← check
```

THE HUMAN EDGE

- › **Collect evidence** — retry history, timing, recent diffs.
- › **Branch by taxonomy** — timing / ordering / environment.
- › **Prune** — kill branches the evidence rules out.
- › **Confirm** — reproduce with the smallest change.

AI-augmented triage

AI is fast at structuring log noise and proposing categories — you still decide.

DATADOG FLAKY TEST MGMT

- › Auto-detects flakiness across the org.
- › **Lifecycle:** Active → Quarantined → Disabled → Fixed.
- › AI assigns a root-cause category (concurrency, timeout, network...).
- › Bits AI can open a fix PR on GitHub.

CHATGPT / CLAUDE ON LOGS

- › Paste the failing log + the test source.
- › Ask for a ranked hypothesis list.
- › Ask for the 3 cheapest next checks.
- › Free-tier friendly — no paid account needed.

A triage prompt you can reuse

Structure the ask so the model prunes like an engineer — not a paragraph of guesses.

log-triage prompt

```
# Prompt
You are a CI flaky-test triager.
Here is a Jest test and its failing log
(3/10 runs). Classify the likely cause as
timing | ordering | environment.

Return:
1. Ranked hypotheses with confidence %
2. The 3 cheapest checks to confirm
3. A minimal patch for the top hypothesis

<<< paste test source + full log here >>>
```

WHY IT WORKS

- › Gives it the taxonomy to map onto.
- › Forces ranking + confidence, not prose.
- › Asks for cheap checks first.
- › You verify before trusting.

Where AI helps — and where it hurts

The seminar's real question. Be specific in your report.

AI ADDS VALUE

- › Structuring 10k lines of log noise fast.
- › Proposing a taxonomy category to start from.
- › Drafting a candidate patch to review.
- › Spotting repeated failure signatures.

AI ADDS NOISE

- › Confident-but-wrong root causes.
- › Hallucinated file/line/config that doesn't exist.
- › Hiding a real bug by quarantining a “flake”.
- › Fix PRs that mask timing with a bigger sleep.

Test impact analysis

Don't run all tests — run the ones this commit can actually break.

THE IDEA

- › **Map** — learn which tests exercise which code.
- › **Predict** — for a diff, pick the subset likely to fail.
- › **Run that subset first** — fail fast, defer the rest.
- › **Payoff** — minutes not hours on large suites.

TOOLS TO WATCH

Launchable

acquired by CloudBees
(2024); on AWS Marketplace

Sealights

quality governance +
test impact analysis

DiffBlue

AI unit-test generation (Java)

Pipeline Postmortem

Audience teams diagnose a broken build from only the YAML + a log snippet.

0:00–0:03

Facilitator hands out the failing log + workflow.yml

0:03–0:13

Teams write a hypothesis tree, 3 levels deep

0:13–0:18

Each team consults an AI tool with the same logs

0:18–0:22

Reveal the real root cause; debrief

0:22–0:25

Each team states one lesson learned

TAKEAWAY AI excels at structuring log noise; humans excel at hypothesis pruning. Reproducible by a peer team in ≤ 25 min.

Failure modes — 3 ways these tools mislead

Your user guide must list these. Tools help until they quietly don't.

- › **Green ≠ correct** — a passing pipeline only proves the tests you wrote ran; missing tests pass silently.
- › **AI confident-but-wrong** — a fluent root-cause with a hallucinated file can send a team down the wrong path for an hour.
- › **Quarantine as a rug** — auto-quarantining a “flake” can hide a genuine, intermittent production bug.

RULE Every AI suggestion is a hypothesis, not a verdict. Validate against live metadata and a real reproduction before you trust it.

WRAP-UP

What to take away

Remember

- A test that isn't automated doesn't exist.
- Order stages cheap → expensive; fail fast.
- Classify flakes: timing / ordering / env.
- AI structures noise; you prune & verify.
- Next: test impact analysis.

READING LIST

Humble & Farley —
Continuous Delivery (Ch 4,8)

Google Testing Blog —
Flaky tests at Google

Datadog — Flaky test
analytics (tech posts)

Deliverables: Tool survey · User guide (≥6 sec, incl. failure modes) · 5–8 min screencast · Postmortem worksheet · Slides.